

8.26

YUV RGB (编码格式)

YUV (适合存储和传输)

- **Y (Luma): 亮度分量。** 包含了图像的灰度信息, **决定了图像的明暗对比和细节。** 人眼对亮度信息最为敏感。
- **UV (CbCr): 色度分量。** 包含了图像的颜色信息, **指定了像素的颜色和饱和度。** 人眼对颜色信息的细微变化不那么敏感。UV部分就可以进行压缩

YUV420P

本程序中的函数可以将YUV420P数据中的Y、U、V三个分量分离开来并保存成三个文件;

```
C++    hhh

1  /**
2   * Split Y, U, V planes in YUV420P file.
3   * @param url   Location of Input YUV file.
4   * @param w     Width of Input YUV file.
5   * @param h     Height of Input YUV file.
6   * @param num   Number of frames to process.
7   *
8   */
9  int simplest_yuv420_split(char *url, int w, int h, int num){
10     FILE *fp=fopen(url, "rb+");
11     FILE *fp1=fopen("output_420_y.y", "wb+");
12     FILE *fp2=fopen("output_420_u.y", "wb+");
13     FILE *fp3=fopen("output_420_v.y", "wb+");
14
15     unsigned char *pic=(unsigned char *)malloc(w*h*3/2);
16
17     for(int i=0; i<num; i++){
18
19         fread(pic, 1, w*h*3/2, fp);
```

```

20         //Y
21         fwrite(pic,1,w*h,fp1);
22         //U
23         fwrite(pic+w*h,1,w*h/4,fp2);
24         //V
25         fwrite(pic+w*h*5/4,1,w*h/4,fp3);
26     }
27
28     free(pic);
29     fclose(fp);
30     fclose(fp1);
31     fclose(fp2);
32     fclose(fp3);
33
34     return 0;
35 }

```

从代码可以看出，如果视频帧的宽和高分别为 w 和 h ，那么一帧YUV420P像素数据一共占用 $w*h*3/2$ Byte的数据。其中前 $w*h$ Byte 存储Y，接着的 $w*h*1/4$ Byte存储U，最后 $w*h*1/4$ Byte存储V。上述调用函数的代码运行后，将会把一张分辨率为 256x256 的名称为lena_256x256_yuv420p.yuv的YUV420P格式的像素数据文件分离成为三个文件：

output_420_y.y：纯Y数据，分辨率为256x256。

output_420_u.y：纯U数据，分辨率为128x128。

output_420_v.y：纯V数据，分辨率为128x128。

注：本文中像素的采样位数一律为8bit。由于1Byte=8bit，所以一个像素的一个分量的采样值占用1Byte。

YUV 444P

YUV444P数据中的Y、U、V三个分量分离开来并保存成三个文件，函数的代码如下所示。

```

1  /**
2   * Split Y, U, V planes in YUV444P file.
3   * @param url  Location of YUV file.

```

```

4  * @param w      Width of Input YUV file.
5  * @param h      Height of Input YUV file.
6  * @param num    Number of frames to process.
7  *
8  */
9  int simplest_yuv444_split(char *url, int w, int h, int num){
10     FILE *fp=fopen(url, "rb+");
11     FILE *fp1=fopen("output_444_y.y", "wb+");
12     FILE *fp2=fopen("output_444_u.y", "wb+");
13     FILE *fp3=fopen("output_444_v.y", "wb+");
14     unsigned char *pic=(unsigned char *)malloc(w*h*3);
15
16     for(int i=0; i<num; i++){
17         fread(pic, 1, w*h*3, fp);
18         //Y
19         fwrite(pic, 1, w*h, fp1);
20         //U
21         fwrite(pic+w*h, 1, w*h, fp2);
22         //V
23         fwrite(pic+w*h*2, 1, w*h, fp3);
24     }
25
26     free(pic);
27     fclose(fp);
28     fclose(fp1);
29     fclose(fp2);
30     fclose(fp3);
31
32     return 0;
33 }

```

，如果视频帧的宽和高分别为 w 和 h ，那么一帧YUV444P像素数据一共占用 $w*h*3$ Byte的数据。其中前 $w*h$ Byte存储Y，接着的 $w*h$ Byte存储U，最后 $w*h$ Byte存储V。上述调用函数的代码运行后，将会把一张分辨率为256x256的名称为lena_256x256_yuv444p.yuv的YUV444P格式的像素数据文件分离成为三个文件：output_444_y.y：纯Y数据，分辨率为256x256。output_444_u.y：纯U数据，分辨率为256x256。output_444_v.y：纯V数据，分辨率为256x256。

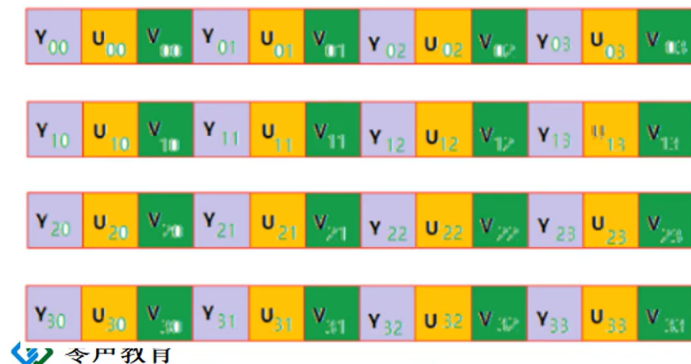
排列方式

3.2 YUV

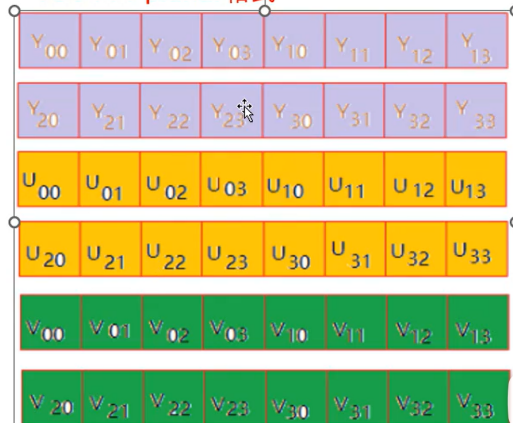
■ YUV是一个比较笼统的说法，针对它的具体排列方式，可以分为很多种具体的格式：

- 打包 (packed) 格式：将每个像素点的Y、U、V分量交叉排列并以像素点为单元连续的存放在同一数组中，通常几个相邻的像素组成一个宏像素 (macro-pixel)
- 平面 (planar) 格式：使用三个数组分开连续的存放Y、U、V三个分量，即Y、U、V分别存放在各自的数组中。

YUV444 packed格式



YUV444 planar格式



- 为什么常见的编码格式都是 平面格式？

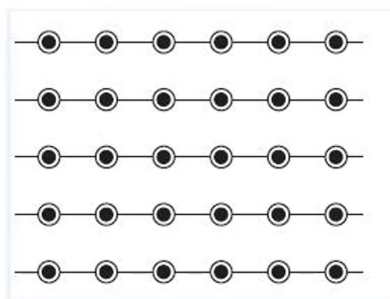
因为在视频压缩时，Y U V是分开做编码的；

YUV采样表示法

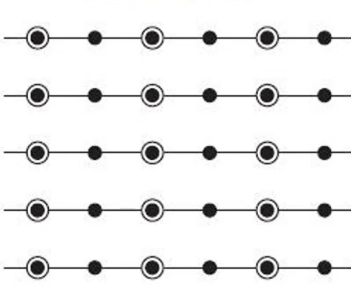
3.2.1 YUV采样表示法

■ YUV采用A:B:C表示法来描述Y,U,V采样频率比例，下图中黑点表示采样像素点Y分量，空心圆表示采样像素点的UV分量。主要分为 YUV 4:4:4、YUV 4:2:2、YUV 4:2:0 这几种常用的类型

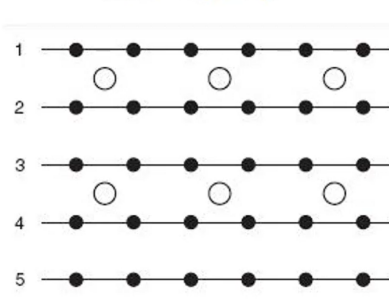
YUV4:4:4采样



YUV4:2:2采样



YUV4:2:0采样

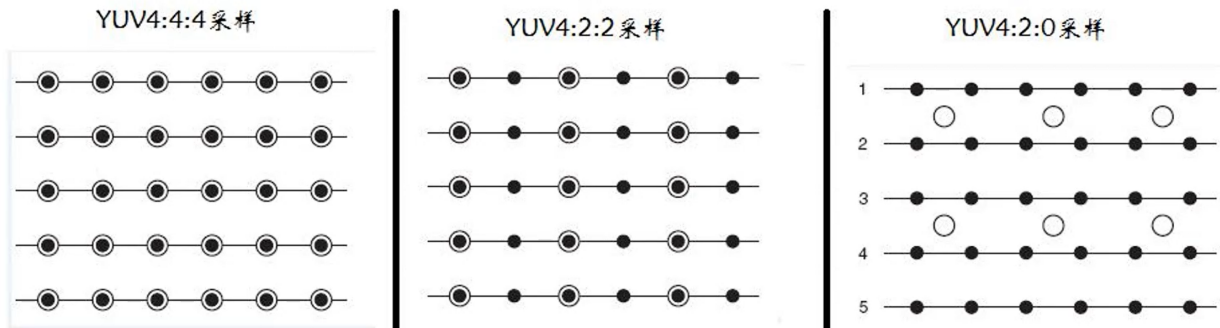


- 4:4:4 表示色度频道没有下采样，即一个Y分量对应着一个U分量和一个V分量。
- 4:2:2 表示 2:1 的水平下采样，没有垂直下采样，即每两个Y分量共用一个U分量和一个V分量。
- 4:2:0 表示 2:1 的水平下采样，2:1 的垂直下采样，即每四个Y分量共用一个U分量和一个V分量。

- 为什么YUV420（采样为4：2：0）这个表示用得最多的？

3.2.1 YUV采样表示法

- YUV采用A:B:C表示法来描述Y,U,V采样频率比例，下图中黑点表示采样像素点Y分量，空心圆表示采样像素点的UV分量。主要分为 YUV 4:4:4、YUV 4:2:2、YUV 4:2:0 这几种常用的类型



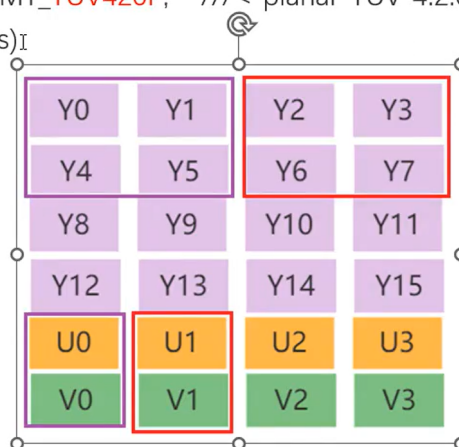
- 4:4:4 表示色度频道没有下采样，即一个Y分量对应着一个U分量和一个V分量。
- 4:2:2 表示 2:1 的水平下采样，没有垂直下采样，即每两个Y分量共用一个U分量和一个V分量。
- 4:2:0 表示 2:1 的水平下采样，2:1 的垂直下采样，即每四个Y分量共用一个U分量和一个V分量。

UV (CbCr): 色度分量。 包含了图像的颜色信息，UV使用同一个值变换也不大

最常见的 YUV420P，也是现在项目中用到的；

3.2.2 YUV数据存储-4:2:0格式-YUV420P

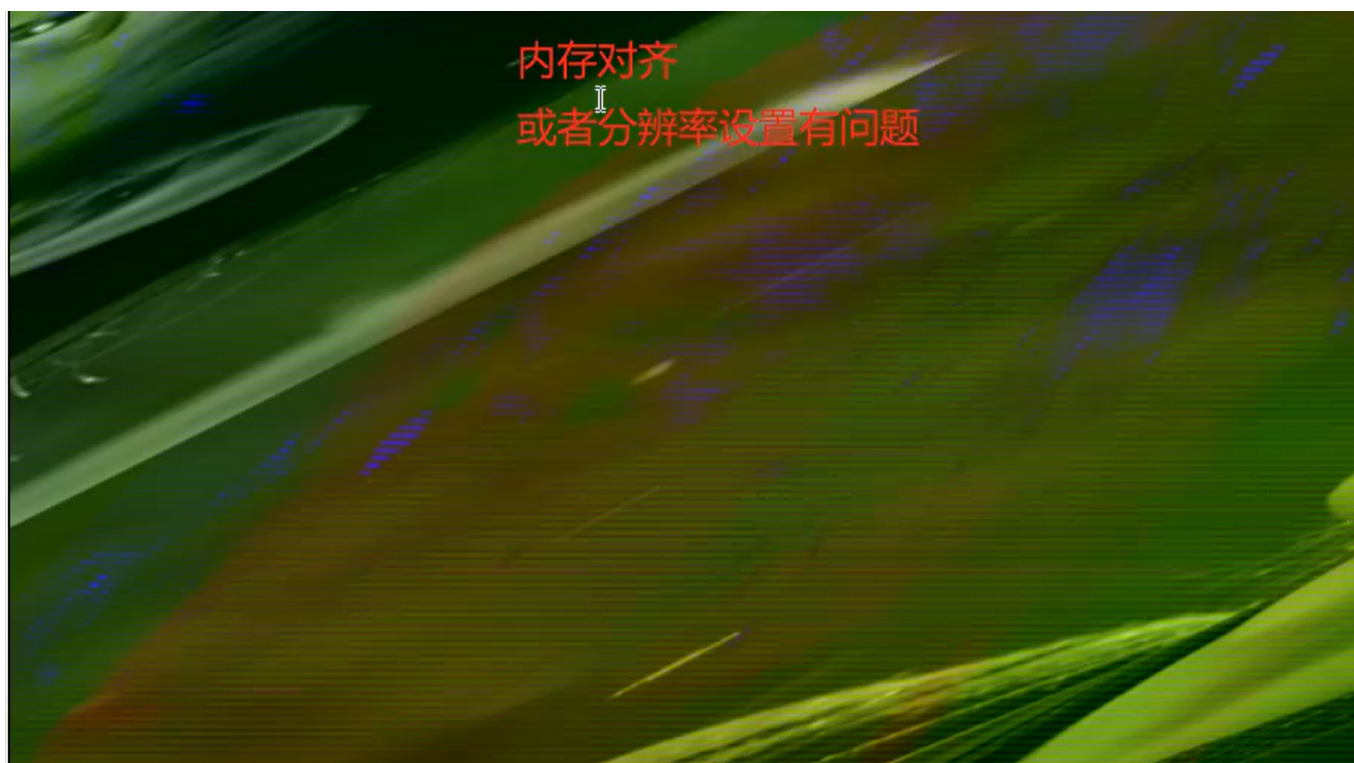
- 比如I420(YUV420P)格式
 - 对应Ffmpeg像素表示AV_PIX_FMT_YUV420P, ///< planar YUV 4:2:0, 12bpp, (1 Cr & Cb sample per 2x2 Y samples)I
 - 该类型为planar格式
 - $(4+1+1)/4 = 1.5$ 字节



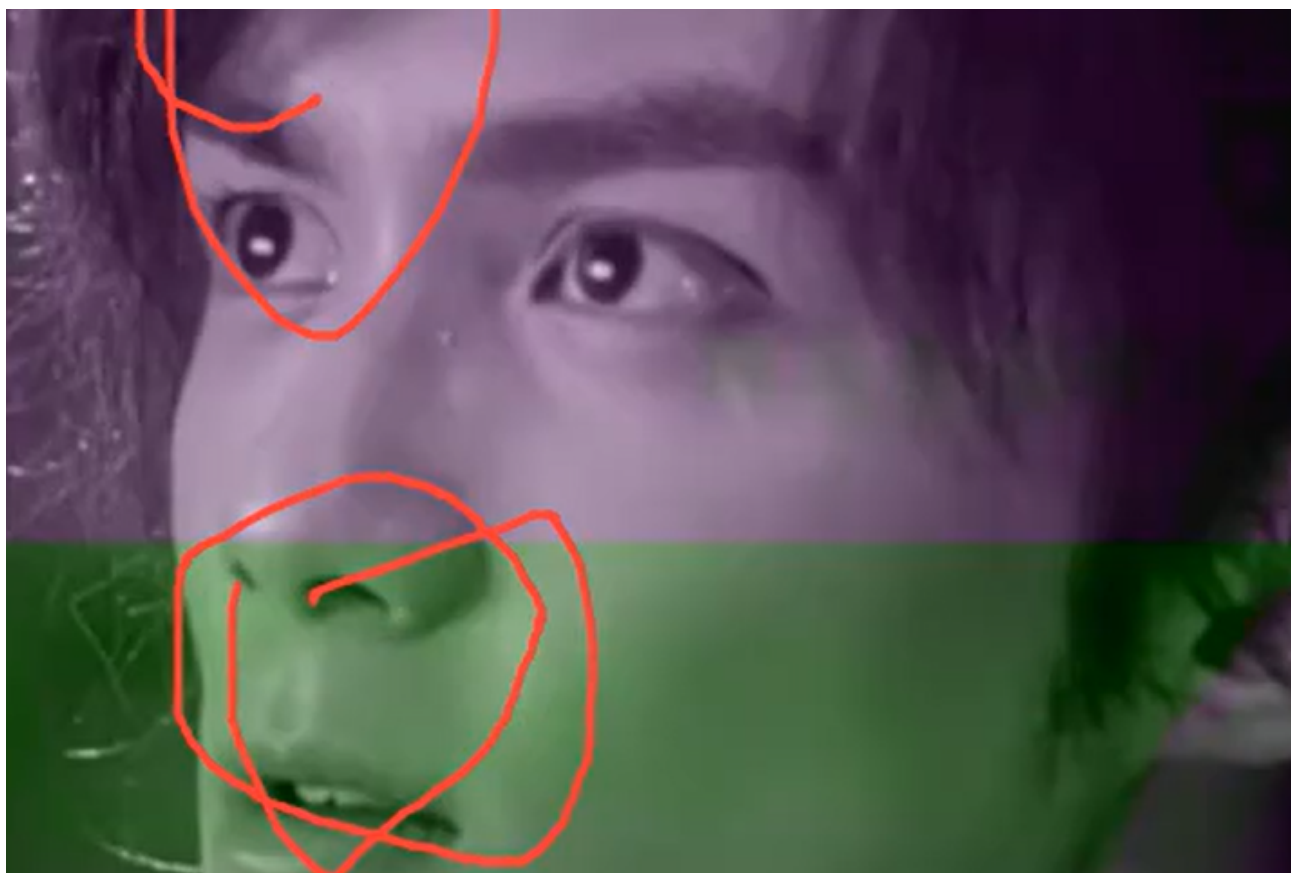
I420(YUV420P)格式

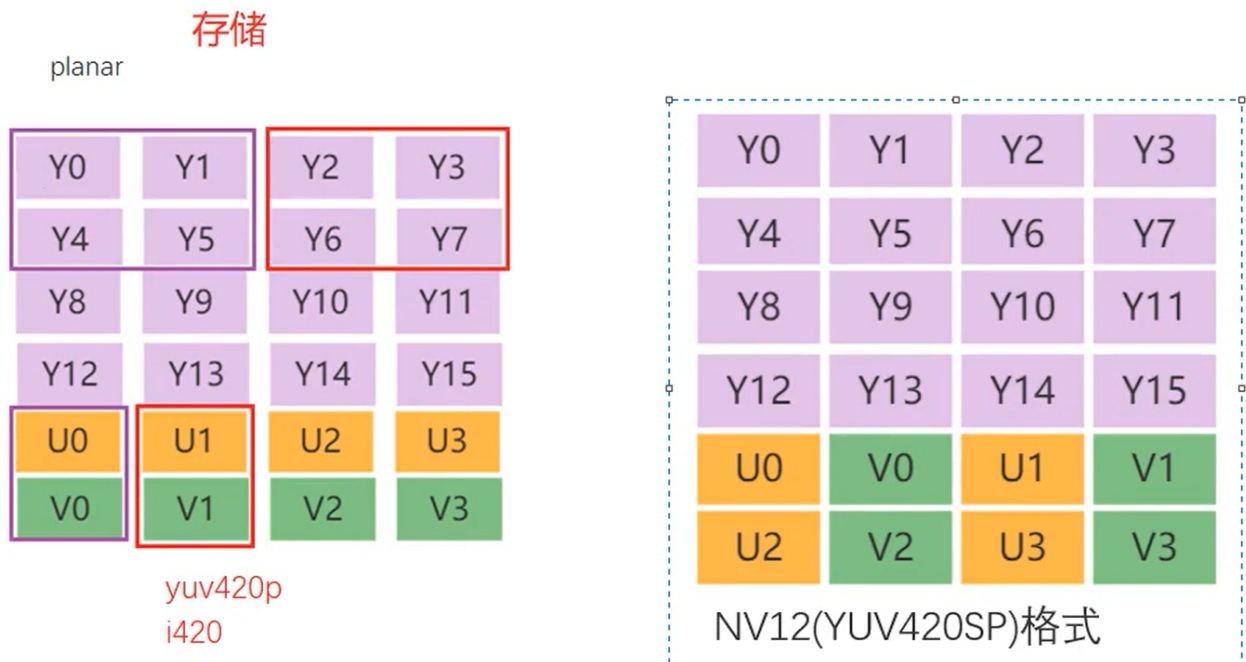
YUV导致的视频问题

- 内存没有对齐或者分辨率选择不当会导致内容斜体



- YUV格式设置不当(实际就是) (eg 设置为 YV12或者NV12等) 会导致颜色显示不正常





很明显 YUV420P 的U V格式与NV12的不同；

YUV420P 的U V 存放是平面格式， U V分开连续存放；

而NV12的存放是打包格式， U V交叉存放；

分辨率

2.2 分辨率-隔行扫描和逐行扫描1

有些网站的分辨率显示是 720i 和 1080i，这个 i 又指的是什么？同样都是1080，1080i 和 1080p¹哪个会更清晰？

i 和 p 是指扫描方式，i 表示隔行扫描(Interlaced)，p 则表示逐行扫描(Progressive)，以 1080 的视频举例：

- 1080i：1920 x 1080 分辨率。隔行扫描模式下的高清图像是隔行显示的。每一个奇数行图像都在每一偶数行图像后面显示出来。比如将 60 帧分成两部分，奇数帧只扫描 1,3,5,7,9... 行，偶数帧只扫描 2,4,6,8,10... 行。理论上人眼是察觉不出来画面不连续，反而是由于视觉残留，能自动将两帧叠加在一起。
- 1080p：1920x1080 分辨率。和 1080i 的区别就在于 1080p 不是隔行扫描，是逐行扫描。每一线都同时表现在画面上，因此比隔行扫描电视更加的平滑。

- i:隔行扫描
- p:逐行扫描

RGB (适合显示)

通过不同强度的红、绿、蓝光相加混合来产生各种颜色。这直接对应了显示器屏幕上每个物理像素点的子像素结构；

可以将RGB24数据中的R、G、B三个分量分离开来并保存成三个文件。函数的代码如下所示。

```
1  /**
2   * Split R, G, B planes in RGB24 file.
3   * @param url   Location of Input RGB file.
4   * @param w     Width of Input RGB file.
5   * @param h     Height of Input RGB file.
6   * @param num   Number of frames to process.
7   *
8   */
9  int simplest_rgb24_split(char *url, int w, int h, int num){
10     FILE *fp=fopen(url, "rb+");
11     FILE *fp1=fopen("output_r.y", "wb+");
12     FILE *fp2=fopen("output_g.y", "wb+");
13     FILE *fp3=fopen("output_b.y", "wb+");
14
15     unsigned char *pic=(unsigned char *)malloc(w*h*3);
16
17     for(int i=0; i<num; i++){
18
19         fread(pic, 1, w*h*3, fp);
20
21         for(int j=0; j<w*h*3; j=j+3){
22             //R
23             fwrite(pic+j, 1, 1, fp1);
24             //G
25             fwrite(pic+j+1, 1, 1, fp2);
26             //B
27             fwrite(pic+j+2, 1, 1, fp3);
28         }
29     }
30
31     free(pic);
32     fclose(fp);
33     fclose(fp1);
34     fclose(fp2);
35     fclose(fp3);
36 }
```



```

37     return 0;
38 }

```

与YUV420P三个分量分开存储不同，RGB24格式的每个像素的三个分量是连续存储的。一帧宽高分别为w、h的RGB24图像一共占用 $w*h*3$ Byte的存储空间。RGB24格式规定首先存储第一个像素的R、G、B，然后存储第二个像素的R、G、B...以此类推。类似于YUV420P的存储方式称为Planar方式，而类似于RGB24的存储方式称为Packed方式。上述调用函数的代码运行后，将会把一张分辨率为500x500的名称为cie1931_500x500.rgb的RGB24格式的像素数据文件分离成为三个文件：

- output_r.y: R数据，分辨率为256x256。
- output_g.y: G数据，分辨率为256x256。
- output_b.y: B数据，分辨率为256x256。

项目中为什么在解码后将 YUV转为RGB?

3.3 RGB和YUV的转换

- 通常情况下RGB和YUV直接的相互转换都是调用接口实现，比如Ffmpeg的swscale或者libyuv等库。

- 主要转换标准是 BT601 和 BT709。

8bit位深的情况下

- TV range是16-235(Y)、16-240(UV)，也叫Limited Range
- PC range是0-255，也叫Full Range
- 而RGB没有range之分，全是0-255

- BT601 TV Range转换公式

YUV(256 级别) 可以从8位 RGB 直接计算：

$$Y = 0.299 * R + 0.587 * G + 0.114 * B;$$

$$U = -0.169 * R - 0.331 * G + 0.5 * B;$$

$$V = 0.5 * R - 0.419 * G - 0.081 * B;$$

反过来，RGB 也可以直接从YUV (256级别) 计算：

$$R = Y + 1.402 (Y-128)$$

$$G = Y - 0.34414 (U-128) - 0.71414 (U-128)$$

$$B = Y + 1.772 (V-128)$$

- 从YUV 转到 RGB 如果值小于0要取0，如果大于255要取255

- YUV:为了在有限的带宽下传输彩色电视信号，并保持与黑白电视的兼容性（黑白电视只需处理Y分量）。利用人眼的特性，可以对色度分量（UV）进行降采样（比如常见的YUV420格式），大幅减少数据量，而人

眼几乎察觉不到画质损失。因此，YUV是视频编码、压缩和传输的理想格式。

- RGB:几乎所有数字显示设备，如LCD、OLED屏幕，其底层硬件都期望接收RGB格式的数据来驱动每个像素点发光。

为啥选择转为RGB32而不是而不是RGB24?

“32”指的是每个像素占用32位（4个字节）的内存空间。这4个字节的分配通常是：

- 1字节 (8 bits) - R (Red)
- 1字节 (8 bits) - G (Green)
- 1字节 (8 bits) - B (Blue)
- 1字节 (8 bits) - Alpha (透明度) 或 填充位 (Unused)

1. 内存对齐：现代计算机CPU（如32位、64位架构）在处理数据时，以4字节或8字节为单位进行读写效率最高；RGB24（每一个像素占3个字节）会导致像素的起始地址可能不在4字节的整数倍上，某些硬件或软件在处理非对齐数据时性能会下降。
2. 硬件加速：很多图形API（如我的OpenGL）和硬件优化指令集在处理32位像素时效率更高，因为这与系统字长匹配得更好。

不在解码线程中间YUV转为RGB格式，而是直接将RGB数据交给OpenGL渲染以提高效率

一开始的方案是采用将接收到的解码后的 YUV420P格式数据直接转为RGB32格式，再转交给OpenGL渲染；

但是效率降低了，会有一定的延迟；

一段1080p（1920x1080）的视频帧，如果使用YUV420格式，其数据量约为： $(1920 * 1080) + (960 * 540) * 2 \approx 2.33 \text{ MB}$ 。如果在CPU上将其转换为RGB32，一帧的数据量会变为： $1920 * 1080 * 4 \approx 7.75 \text{ MB}$ 。这意味着每一帧都需要额外进行 $\sim 2.33\text{MB} \rightarrow \sim 7.75\text{MB}$ 的数据转换和传输（从CPU内存上传到GPU显存）；

而OpenGL的强大之处在于**并行计算**。YUV到RGB的转换是一个可以完美并行化的任务（每个像素的计算都是独立的）。将这项工作交给**GPU**（通过着色器Shader），可以让成千上万个核心同时工作，效率极高，且完全解放了CPU。在CPU端做转换，是串行处理，浪费了GPU的计算能力。

RGB和YUV转换导致的绿屏问题（重点！！）

3.3 RGB和YUV的转换

- 通常情况下RGB和YUV直接的相互转换都是调用接口实现，比如Ffmpeg的swscale或者libyuv等库。

I

- 主要转换标准是 BT601 和 BT709。

8bit位深的情况下

- TV range是16-235(Y)、16-240(UV)，也叫Limited Range
- PC range是0-255，也叫Full Range
- 而RGB没有range之分，全是0-255

- BT601 TV Range转换公式

YUV(256 级别) 可以从8位 RGB 直接计算：

$$Y = 0.299 * R + 0.587 * G + 0.114 * B;$$

$$U = -0.169 * R - 0.331 * G + 0.5 * B;$$

$$V = 0.5 * R - 0.419 * G - 0.081 * B;$$

反过来，RGB 也可以直接从YUV (256级别) 计算：

$$R = Y + 1.402 (Y-128)$$

$$G = Y - 0.34414 (U-128) - 0.71414 (U-128)$$

$$B = Y + 1.772 (V-128)$$

- 从YUV 转到 RGB 如果值小于0要取0，如果大于255要取255

3.3 RGB和YUV的转换-为什么解码出错显示绿屏

因为解码失败时YUV分量都填为0值，然后根据公式：

$$R = 1.402 * (-128) = -126.598$$

$$G = -0.34414 * (-128) - 0.71414 * (-128) = 44.04992 + 91.40992 = 135.45984$$

$$B = 1.772 * (-128) = -126.228$$

RGB 值范围为[0, 255]， 所以最终的值为：

$$R = 0$$

$$G = 135.45984$$

$$B = 0$$

此时只有G分量有值所以为绿色。



YUV -> RGB